```
// var a=10;
// a=2;
// console.log(a); //2

// a=2;
// var a;
// console.log(a); //2

// let a=10;
// a=10;
// console.log(a); //10

// a=10;
// let a;
// console.log(a); //10

// const a=10;
// a=10;
// console.log(a); //TypeError: Assignment to constant variable.

// const a;
// a=10;
// console.log(a); //SyntaxError: Missing initializer in const declaration

// a=10;
// const a;
// console.log(a); //SyntaxError: Missing initializer in const declaration

//console.log(5=="5");//true
//console.log(5==="5");//false

//-----------------------------------------------------------------

// let a=prompt("enter the input:");
// console.log(a + is + typeof(a)); //works only in browser not in terminal

// let a=10;
// let b="name";
// let c=true;
// let d="";
//console.log(typeof(a));//number
//console.log(typeof(b));//string
//console.log(typeof(c));//boolean
//console.log(typeof(d));//string

//----------------------------------------

// let a=parseInt(prompt("enter the value:"));
// let b=parseInt(prompt("enter the value:"));
// console.log("sum is " + (a+b));
// console.log("Difference is " + (a-b));
// console.log("product is " + (a*b));
// console.log("quotient is " + (a/b));
```

```
// console.log("remainder " + (a%b));

//---------------------------------

// let a=parseInt(prompt("enter the value:"));
// let b=parseInt(prompt("enter the value:"));
// if(a>b){
//     console.log("a is greater");
// }else{
//     console.log("b is greater");
// }

//----------------------

// let a=parseInt(prompt("enter the value:"));
// let b=parseInt(prompt("enter the value:"));
// let temp;
// temp=a;
// a=b;
// b=temp;
// console.log(a);
// console.log(b);

//------------------------

// let a="123";
// console.log(typeof(a));//string
// let num=Number(a);
// console.log(typeof(num));//number

                                //******CONTROL FLOW******

// let a=parseInt(prompt("enter the value:"));
// if(a%2==0){
//     console.log("even number");
// }else{
//     console.log("odd number");
// }

//----------


// let a=parseInt(prompt("enter the value:"));
// if(a>0){
//     console.log("positive number");
// }else if(a<0){
//     console.log("negative number");
// }else{
//     console.log("zero");
// }

//---------

// let a=parseInt(prompt("enter the value:"));
```

```
// let b=parseInt(prompt("enter the value:"));
// let c=parseInt(prompt("enter the value:"));
// if(a>b){
//     if(a>c){
//         console.log("a is largest number");
//     }
// }else if(b>c){
//         console.log("b is greater");
// }else{
//     console.log("c is greater");
// }

//---------------

// let day=parseInt(prompt("enter the value:"));
// let dayName;
// switch(day){
//    case 1:
//      dayName="Monday";
//      break;
//     case 2:
//      dayName="Tuesday";
//      break;
//     case 3:
//      dayName="Wednesday";
//      break;
//     case 4:
//      dayName="Thursady";
//      break;
//     case 5:
//      dayName="Friday";
//      break;
//     case 6:
//      dayName="Saturday";
//      break;
//     case 7:
//      dayName="Sunday";
//      break;
//     default:
//       dayName="enter a valid number";
// }
// console.log(dayName);


// let num=parseInt(prompt("enter the value:"));
// for(let i=0;i<=num;i++){
//     console.log(i)
// }


// let i=10;
// while(i>=1){
// console.log(i);
//     i--;
```

```
// }


// for(let i=1;i<=5;i++){
//     let star="";
//     for(let j=1;j<=i;j++){
//          star += "* ";
//     }
//     console.log(star);
// }

//-------------------

// for(let i=1;i<=50;i++){
//     if(i%2==0){
//         console.log(i);
//     }
// }

//-----------

// let n=parseInt(prompt("enter the value:"));
// let sum=(n*(n+1))/2;
// console.log(sum);

//OR

//using for loop
// let n=parseInt(prompt("enter the value:"));
// let sum=0;
// for(let i=1;i<=n;i++){
//     sum+=i;
// }
// console.log(sum);

//OR

//using while loop
// let n=parseInt(prompt("enter the value:"));
// let sum=0;
// let i=1;
// while(i<=n){
//     sum+=i;
//     i++;
// }
// console.log(sum);

//--------------

// let num=parseInt(prompt("enter the value:"));
// for(let i=1;i<=10;i++){
//     console.log(num,"*",i,"=",num*i);
// }
```

```
                                //******FUNCTIONS *****

// function sum(a,b){
//     return a+b;
// }
// console.log(sum(2,3));

//--------
// function myfunction(){
//     console.log("Hello World");
// }
// myfunction();
// myfunction();

//---------
// function square(num){
//     return num*num;
// }
// console.log(square(5));

//--------
// function evenodd(num){
//     return (num%2===0)? "even number":"odd number";
// }
// console.log(evenodd(4));

//---------
// function largest(a,b){
//     return (a>b)? a+" is greater":b+" is greater";
// }
// console.log(largest(4,5));

//-----------ARROW FUNCTION
// const sum=(a,b)=>a+b;
// console.log(sum(2,3));

//---------
// const multiply=(a,b)=>a*b;
// console.log(multiply(2,9));

//---------
// const cube=(num)=>num**3;
// console.log(cube(2));

//--------Parameters & Return
// function avg(a,b,c){
//     return (a+b+c)/3;
// }console.log(avg(1,2,4));

//--------------
// function fact(num){
//     if(num<0) return "negative numbers are not valid";
//     let result=1;
//     for(let i=2;i<=num;i++){
```

```
//         result*=i;
//     }
//     return result;
// }
// console.log(fact(2));

                        //*************ARRAYS

// const myArray=[1,2,3,4,5];
// console.log(myArray);
//-------------------
// const myArray=["john","virat","rohit","smriti","shreyanka"];
// console.log(myArray);
// console.log(myArray.length);
//myArray.push("siraj");
// console.log(myArray);//[ 'john', 'virat', 'rohit', 'smriti', 'shreyanka', 'siraj' ]
// myArray.pop();
// console.log(myArray);//[ 'john', 'virat', 'rohit', 'smriti', 'shreyanka' ]
// myArray.shift();
// console.log(myArray);//[ 'virat', 'rohit', 'smriti', 'shreyanka' ]
// myArray[1]="john";
// console.log(myArray);//[ 'virat', 'john', 'smriti', 'shreyanka' ]
//------------------
// let nums=[1,2,3,4,5];
// let newArray=nums.map((val)=>val+val);
// console.log(nums);
// console.log(newArray);
//-----------------
// let nums=[1,2,3,4,5,6,7,8,9,10];
// let newArray=nums.filter((val)=>val%2===0)
//console.log(nums);//[1,2,3,4,5,6,7,8,9,10]
//console.log(newArray); //[ 2, 4, 6, 8, 10 ]
//----------------
// let nums=[1,2,3,4,5];
// let newArray=nums.reduce((prev,curr)=>prev+curr);
// console.log(newArray);
//------------
// let nums=[1,10,18,2,8];
// let newArray=nums.reduce((prev,curr)=>prev>curr?prev:curr);
// console.log(newArray);
//----------
// let myArray=["apple","ball","cat","dog"];
// let newArray=myArray.map((val)=>val.toUpperCase());
// console.log(newArray);//[ 'APPLE', 'BALL', 'CAT', 'DOG' ]
//-----------
// let nums=[2,4,6,3,8,9];
// let newArray=nums.map((val)=>val>5?val+10:val);
// console.log(newArray);//[ 2, 4, 16, 3, 18, 19 ]
//-----------
// let nums=[10, 55, 80, 30, 90];
// let newArray=nums.filter((val)=>val>50);
// console.log(newArray);//[ 55, 80, 90 ]
//-----------
// let myArray=["apple", "banana", "orange", "grapes"];
```

```
// let newArray=myArray.filter((val)=>val[0] === 'a' || val[0] === 'e' || val[0] === 'i' ||
//     val[0] === 'o' || val[0] === 'u' ||val[0] === 'A' || val[0] === 'E' || val[0] === 'I'
||
//     val[0] === 'O' || val[0] === 'U');
// console.log(newArray);//[ 'apple', 'orange' ]
//---------
// let myArray=["apple", "banana", "orange", "grapes"];
// let newArray=myArray.filter((val)=>"aeiouAEIOU".includes(val[0]));
// console.log(newArray);//[ 'apple', 'orange' ]
//-----------
// let nums = [1, 2, 3, 4, 6, 7];
// let countEven = nums.reduce((count, val) => {
//     return val % 2 === 0 ? count + 1 : count;
// },0);
// console.log(countEven);//3

                            //********** OBJECTS
// const student={
//     name:"John",
//     age:21,
//     course:"Fullstack"
// };
// console.log(student);
// console.log(student.age);
// console.log(student.course);
// student.id=101;
// console.log(student);
// student.name="Virat";
// console.log(student);

//Loop Through an Object
// let text="";
// for(let x in student){
//     text+=student[x]+" ";
// };
// console.log(text);

// const myArray=Object.values(student);
// console.log(myArray); //result is in array

// let text="";
// for(let [marks,value] of Object.entries(student)){
//     text+=`${marks}:${value} \n`;
// };
// console.log(text);

//Nested Object Loop
// const student={
//     name:"John",
//     age:21,
//     course:{
//         frontend:"JS",
//         backend:"NODEJS",
//     }
```

```
// };

//const myArray=Object.values(student);
//console.log(myArray);//[ 'John', 21, { frontend: 'JS', backend: 'NODEJS' } ]

// let users = {
//   user1: { age: 18 },
//   user2: { age: 25 },
//   user3: { age: 16 }
// };

//Count Properties
// let count=0;
// for(let user of Object.values(users)){
//     if(user.age>=18){
//         count++;
//     }
// }
// console.log(count);

//Check Property Exists
// let employee = {
//   id: 101,
//   name: "Ravi",
//   salary: 40000
// };
//  if("department" in employee){
//     console.log("department exists");
//  }else{
//     console.log("department does not exists");
//  }

//check property exists
// let employee = {
//   id: 101,
//   name: "Ravi",
//   salary: 40000
// };
//  if(employee.hasOwnProperty("department")){
//     console.log("department exists");
//  }else{
//     console.log("department does not exists");
//  }

//Count Number of Properties
// let book = {
//   title: "JS Basics",
//   author: "John",
//   pages: 250,
//   price: 399
// };
// let count=0;
// for(let [key,value] in Object.entries(book)){
//     count++;
```

```
// }
// console.log(count);
//OR
// let count=Object.entries(book).length;
// console.log(count);

//Sum All Numeric Values
// let marks = {
//   math: 80,
//   science: 70,
//   english: 90
// };
// let count=0;
// for(let x of Object.values(marks)){
//     count+=x;
// }
// console.log(count);//240

//Delete Property Based on Condition***
// let products = {
//   p1: 0,
//   p2: 200,
//   p3: 0,
//   p4: 500
// };
// for(let [key,values] of Object.entries(products)){
//     if(values===0){
//         delete products[key];
//     }
// }
// console.log(products);

//Print Only String Values
// let data = {
//   name: "Anita",
//   age: 22,
//   city: "Mumbai",
//   isStudent: true
// };
// for(let [key,value] of Object.entries(data)){
//     if(typeof value=="string"){
//         console.log(key+":"+value);
//     }
// }

//Find Highest Value
// let salaries = {
//   Ram: 30000,
//   Shyam: 45000,
//   Mohan: 25000
// };
// let maxvalue=0;
// let maxperson="";
// for(let [person,salary] of Object.entries(salaries)){
```

```
//     if(salary>maxvalue){
//         maxvalue=salary;
//         maxperson=person;
//     }
// }
// console.log(maxperson);

//Convert Object to Array Manually Without using Object.values()
// let obj = { a: 1, b: 2, c: 3 };
// let arr=[];
// for(let key in obj){
//     arr.push(obj[key]);
// }
// console.log(arr);
//OR

// let obj = { a: 1, b: 2, c: 3 };
// let arr=[];
// let i=0;
// for(let key in obj){
//     arr[i]=obj[key];
//     i++;
// }
// console.log(arr);//[ 1, 2, 3 ]

//let obj = { a: 1, b: 2, c: 3 };
// let arr=[];
// let i=0;
// for(let key in obj){
//     arr[i]=key;
//     i++;
// }
//console.log(arr);//[ 'a', 'b', 'c' ]

//Count Boolean value equal to true
// let settings = {
//   darkMode: true,
//   notifications: false,
//   autoSave: true,
//   location: false
// };
// let count=0;
// for(let value of Object.values(settings)){
//   if(value===true){
//     count++;
//   }
// }
// console.log(count);

//multiply all numeric numbers
// let numbers = {
//   a: 2,
//   b: 3,
//   c: 4
```

```
// };
// let product=1;
// for(let value of Object.values(numbers)){
//    product*=value;
// }
// console.log(product);

//Create New Object with Doubled Values
// let prices = {
//   apple: 50,
//   banana: 20,
//   mango: 30
// };
// let newObject={};
// for(let [key,value] of Object.entries(prices)){
//    newObject[key]=value*2;
// }
// console.log(newObject); //{ apple: 100, banana: 40, mango: 60 }

//Filter Object Based on Condition
// let users = {
//   user1: { active: true, age: 20 },
//   user2: { active: false, age: 25 },
//   user3: { active: true, age: 16 },
//   user4: { active: true, age: 30 }
// };
// let newObject={};
// for(let [key,value] of Object.entries(users)){
//     if(value.active===true && value.age>=18){
//      newObject[key]=value;
//     }
// }
// console.log(newObject);//{ user1: { active: true, age: 20 }, user4: { active: true, age:
30 } }

//Find Longest String Value
// let words = {
//   a: "apple",
//   b: "banana",
//   c: "kiwi"
// };
// let longestword="";
// let count=0;
// for(let value of Object.values(words)){
//     if(value.length>count){
//         count=value.length;
//         longestword=value;
//     }
// }
// console.log(count,longestword);

//Check If All Values Are Numbers
// let data = {
//   x: 10,
```

```
//   y: 20,
//   z: "30"
// };
// for(let value of Object.values(data)){
//     if(typeof value==="number"){
//         console.log("type of all values are numbers");
//     }else{
//         console.log("type of all values are not numbers");

//     }
// }
                //  OUTPUT:type of all values are numbers
                //type of all values are numbers
                //type of all values are not numbers

// let allNumbers=true;
// for(let values of Object.values(data)){
//     if(typeof values!=="number"){
//         allNumbers=false;
//     }
// }
// console.log(allNumbers);//false

//Count Nested Properties
// let company = {
//   HR: { employees: 5 },
//   IT: { employees: 10 },
//   Sales: { employees: 7 }
// };
// let count=0;
// for(let values of Object.values(company)){
//     count+=values.employees;
// }
// console.log(count);

//SAME IN DIFFERENT WAY
// let count = 0;
// for (let dept of Object.values(company)) {
//     for (let value of Object.values(dept)) {
//         if (typeof value === "number") {
//             count += value;
//         }
//     }
// }
// console.log(count);

//Reverse Key-Value Pairs
// let obj = {
//   a: 1,
//   b: 2,
//   c: 3
// };
// let newObject={};
// for(let [key,value] of Object.entries(obj)){
```

```
//     newObject[value]=key;
// }
// console.log(newObject);//{ '1': 'a', '2': 'b', '3': 'c' }

//Find Duplicate Values
// let data = {
//   a: 10,
//   b: 20,
//   c: 10,
//   d: 30
// };
// for(let value of Object.values(data)){
//   if()
// } //not solved

//Deep Nested Loop
// let students = {
//   s1: { marks: { math: 80, science: 70 } },
//   s2: { marks: { math: 60, science: 90 } }
// };
// let total=0;
// for(let student of Object.values(students)){
//     for(let marks of Object.values(student)){
//         for(let [key,value] of Object.entries(marks)){
//             if(typeof value==="number"){
//                 total+=value;
//             }
//         }
//     }
// }
//OR
// for (let student of Object.values(students)) {
//     for (let value of Object.values(student.marks)) {
//         total += value;
//     }
// }
// console.log(total);//300

                        //************** METHODS
//Create an Object Method
// let calculator={
//     math:95,
//     add:function(a,b){
//         return a+b;
//     }
// }
// console.log(calculator.add(10,8));

//Use this Keyword
// let person={
//     name:"John",
//     age:"21",
//     introduce(){
//         return `Hi, my name is ${this.name} and I am ${this.age} years old`
```

```
//     }
// }
// console.log(person.introduce());//Hi, my name is John and I am 21 years old

//Method Access
// let mobile = {
//   brand: "Samsung",
//   price: 20000,
//   showPrice: function() {
//     return this.price;
//   }
// };
// console.log(mobile.showPrice());

//Update Object Property Using Method
// let bankAccount={
//     balance:80000,
//     deposit(amount){
//         this.balance+=amount;
//         return this.balance
//     }
// }
// console.log(bankAccount.deposit(2000));//82000
// console.log(bankAccount.balance);//82000

//ARROW Function does not have its own this

//Method Returning Object
// let counter={
//     count:0,
//     increment(){
//     this.count+=1;
//     return this;
//     }
// }
// console.log(counter.increment().count);

//Method with Condition
// let employee={
//     name:"John",
//     salary:80000,
//     getBonus(){
//         if(this.salary<30000){
//           this.salary+=5000;
//           return this.salary;
//          }else if(this.salary>=30000){
//             this.salary+=10000;
//           return this.salary;
//          }
//     }
// }
// console.log(employee.getBonus());//90000

//Method Calling Another Method
```

```
// let order = {
//     items: [
//         { name: "Apple", price: 30, quantity: 2 },
//         { name: "Banana", price: 10, quantity: 5 },
//     ],
//     calculatePrice(){
//         let price=0;
//         for(let item of this.items){
//             price+=item.price*item.quantity;
//         }
//         return price;
//     },
//     printBill(){
//         const total=this.calculatePrice();
//         return (`Total Bill: ${total}`);
//     }
// };
// console.log(order.printBill());//110

//Loop Inside Method
// let classroom={
//     marks:[60,70,80,90],
//     totalMarks(){
//         let total=0;
//         for(let mark of this.marks){
//             total+=mark;
//         }
//         return total;
//     },
//  averageMarks(){
//     return this.totalMarks()/this.marks.length;
//  }
//  };
// console.log(classroom.totalMarks());//300
// console.log(classroom.averageMarks());//75

// let user={
//     isLoggedIn: false,
//     login(){
//         this.isLoggedIn= true;
//     },
//     logout(){
//         this.isLoggedIn= false;
//     }
// }
// // console.log(user.isLoggedIn);//false
// user.login();
// console.log(user.isLoggedIn); //true
// user.logout();
// console.log(user.isLoggedIn);//false

//Method Accessing Another Method
// let student={
//     scores:[80,75,90],
```

```
//     gradeRules: {
//         A: 90,
//         B: 80,
//         C: 70
//     },
//     marks(){
//         let total=0;
//         for(let value of Object.values(this.scores)){
//             total+=value;
//         }
//         return total;
//     },
//     grade(){
//         let totalMarks=this.marks();
//         let avg=totalMarks/this.scores.length;
//         if(avg>=this.gradeRules.A) return "A";
//         else if(avg>=this.gradeRules.B) return "B";
//         else if(avg>=this.gradeRules.C) return "C";
//     }
// }
// console.log(student.grade());//B

//Method Using this
// let rectangle={
//     length:10,
//     breadth:20,
//     area(){
//         return this.length*this.breadth;
//     }
// }
// console.log(rectangle.area());//200

//Toggle Boolean Property
// let settings={
//     darkMode : false,
//     toggleDarkMode(){
//         this.darkMode=!this.darkMode;
//         return this.darkMode;
//     }
// };
// console.log(settings.toggleDarkMode());//true
// console.log(settings.toggleDarkMode());//false
// console.log(settings.toggleDarkMode());//true

//Nested Object Method
// let user = {
//   name: "Ravi",
//   address: { city: "Mumbai", pin: 400001 },
//   fullAddress: function() {
//     return this.address.city+"-"+this.address.pin;
//   }
// };
// console.log(user.fullAddress());//Mumbai-400001
```

```
// let account={
//     balance:1000,
//     deposit(amount){
//         this.balance= this.balance+amount;
//         return this.balance;
//     },
//     withdraw(amount){
//         if(amount<=this.balance){
//             this.balance-=amount;
//             return this.balance;
//         }else{
//             return "Insufficient Funds";
//         }
//     }
// };
// console.log(account.deposit(9000));//10000
// console.log(account.withdraw(8000));//2000
// console.log(account.withdraw(3000));//Insufficient Funds

//Count Frequency of Numbers
// let arr=[1,2,2,3,3,3,4];
// let freq={};
// for(let i=0;i<arr.length;i++){
//     let num=arr[i];
//     if(freq[num]){
//         freq[num]+=1;
//     }else{
//         freq[num]=1;
//     }
// }
// console.log(freq);//{ '1': 1, '2': 2, '3': 3, '4': 1 }

           //*******CLASSES
//create a class
// class Student{
//     constructor(name,rollno,course){
//         this.name=name;
//         this.rollno=rollno;
//         this.course=course;
//     }
// }
// let student1=new Student("John",101,"CS");
// console.log(student1);//Student { name: 'John', rollno: 101, course: 'CS' }

//Constructor
// class Car{
//     constructor(brand,price){
//         this.brand=brand;
//         this.price=price;
//     }
// }
// let Car1=new Car("BMW",430000);
// console.log(Car1);//Car { brand: 'BMW', price: 430000 }
```

```
//class method
// class Calculator{
//     multiply(a,b){
//         return a*b;
//     }
// }
// let mult=new Calculator();
// console.log(mult.multiply(2,9));//18

//Use of this in Class
// class Person{
//     constructor(name,age){
//         this.name=name;
//         this.age=age;
//     }
//     details(){
//         return (`Name:${this.name},Age:${this.age}`);
//     }
//     isAdult(){
//         if(this.age>=18){
//             return "Adult";
//         }else{
//             return "Minor";
//         }

//     }
// }
// let Person1=new Person("John",21);
// console.log(Person1.details());//Name:John,Age:21
// console.log(Person1.isAdult());//Adult

//Multiple Objects from One Class
// class Employee{
//      constructor(name,id,salary){
//         this.name=name;
//         this.id=id;
//         this.salary=salary;
//      }
// }
// let employee1=new Employee("John",101,45000);
// let employee2=new Employee("Virat",102,85000);
// console.log(employee1);
// console.log(employee2);

//Update Property Using Method
// class BankAccount{
//     constructor(balance=0){
//          this.balance=balance;
//     }
//     deposit(amount){
//         this.balance+=amount;
//         return this.balance;
//     }
//     getBalance(){
```

```
//         return this.balance;
//     }
// }
// let account=new BankAccount();
// console.log(account.deposit(3000));//3000
// console.log(account.deposit(3000));//6000

//Class with Condition Logic
// class Result{
//     constructor(marks){
//         this.marks=marks;
//     }
//     getGrade(){
//         if(this.marks>=80){
//             return "A";
//         }else if(this.marks>=60){
//             return "B";
//         }else{
//             return "C";
//         }
//     }
// }
// let result1=new Result(90);
// console.log(result1.getGrade());
// let result2=new Result(60);
// console.log(result2.getGrade());

//Inheritance (Basic)
// class Animal{
//     sound(){
//         return "Anima sound";
//     }
// }
// class Dog extends Animal{
//     sound(){
//         return "Bow Bow";
//     }
// }
// let a =new Animal();
// let d=new Dog();
// console.log(a.sound());//Anima sound
// console.log(d.sound());//Bow Bow

//Getter Method
// class Rectangle{
//     constructor(length,breadth){
//         this.length=length;
//         this.breadth=breadth;
//     }
//     get area(){
//         return this.length*this.breadth;
//     }
// }
// let calc1=new Rectangle(20,10);
```

```
// console.log(calc1.area);//200

// class User{
//     constructor(username,password){
//         this.username=username;
//         this.password=password;
//     }
//     login(inputPassword){
//         if(inputPassword===this.password){
//             return "Login successful";
//         }else{
//             return "Invalid password";
//         }
//     }
// }
// let user1=new User("John","john123");
// console.log(user1.login("123"));//Invalid password

                    //******** SUPER
//Basic Inheritance with super
// class Person{
//     constructor(name,age){
//         this.name=name;
//         this.age=age;
//     }
// }
// class Student extends Person{
//     constructor(name,age,rollno){
//         super(name,age);
//         this.rollno=rollno;
//     }
//     getDetails(){
//     return (`Name:${this.name},Age:${this.age},RollNo:${this.rollno}`);
//     }
// }
// let s1=new Student("John",21,101);
// console.log(s1.getDetails());//Name:John,Age:21,RollNo:101

//Method Override Using super
// class Vehicle{
//     constructor(brand){
//         this.brand=brand;
//     }
//     start(){
//         console.log("Vehivle is starting");
//     }
// }
// class Car extends Vehicle{
//     start(){
//         super.start();
//         console.log("Car is ready to drive");
//     }
// }
// let myCar = new Car("Toyota");
```

```
// myCar.start();//Vehivle is starting
              //Car is ready to drive

//Constructor with super
// class Person{
//     constructor(name,age){
//         this.name=name;
//         this.age=age;
//     }
// }
// class Employee extends Person{
//     constructor(name,age,salary){
//         super(name,age);
//         this.salary=salary;
//     }
//     getInfo(){
//         console.log(`name:${this.name},Age:${this.age},Salary:${this.salary}`);
//     }
// }
// let emp1=new Employee("John",21,45000);
// emp1.getInfo();//name:John,Age:21,Salary:45000

//Calling parent method
// class A{
//     greet(){
//         console.log("Hello from A");
//     }
// }
// class B extends A{
//     greet(){
//         super.greet();
//         console.log("Hello from B");
//     }
// }
// let b1=new B();
// b1.greet();//Hello from A
           //Hello from B

//Constructor + method combo
// class Shape{
//     constructor(name){
//         this.name=name;
//     }
//     describe(){
//         console.log(`This is a ${this.name}`);
//     }
// }
// class Rectangle extends Shape{
//     constructor(name,width,height){
//         super(name);
//         this.width=width;
//         this.height=height;
//     }
//     describe(){
```

```
//         super.describe();
//         console.log(`Width:${this.width},Height:${this.height}`);
//     }
// }
// let rect=new Rectangle("John",10,20);
// rect.describe(); //This is a John
                    //Width:10,Height:20

//Multi-level inheritance
// class Animal{
//     speak(){
//         console.log("Animal speaks");
//     }
// }
// class Mammal extends Animal{
//     speak(){
//         super.speak();
//         console.log("Mammal speaks");
//     }
// }
// class Dog extends Mammal{
//     speak(){
//         super.speak();
//         console.log("Dog barks");
//     }
// }
// let d1=new Dog();
// d1.speak();  //Animal speaks
               // Mammal speaks
               // Dog barks

//Multi-Level Inheritance and constructor chaining
// class User{
//     constructor(username){
//         this.name=username;
//     }
//     getRole(){
//         console.log(`Role: user, Name: ${this.name}`);
//     }
// }
// class Admin extends User{
//     constructor(username){
//         super(username);
//     }
//     getRole(){
//         console.log(`Role: Admin, Name: ${this.name}`);
//     }
// }
// class SuperAdmin extends Admin{
//     constructor(username){
//         super(username);
//     }
//     getRole(){
//         console.log(`Role: SuperAdmin, Name: ${this.name}`);
```

```
//     }
// }
// let role=new SuperAdmin("John");
// role.getRole();//Role: SuperAdmin, Name: John

//Inheritance with Calculation Logic
// class Shape{
//     constructor(name){
//         this.name=name;
//     }
//     area(){
//     return 0;
//     }
// }
// class Rectangle extends Shape{
//      constructor(name,length,breadth){
//         super(name);
//         this.length=length;
//         this.breadth=breadth;
//      }
//      area(){
//         return this.length*this.breadth;
//      }
// }
// let a=new Rectangle("abc",10,20);
// console.log("Area:", a.area());//200

                    //SETS**********
//Remove Duplicates
// let arr=[1,2,2,3,2,4,4,5];
// let newarr=[...new Set(arr)];
// console.log(newarr);  //[ 1, 2, 3, 4, 5 ]

//Count Unique Elements
// let arr=[1,2,2,3,2,4,4,5];
// let newarr=[...new Set(arr)];
// console.log(newarr);
// let uniquecount=new Set(arr).size;
// console.log(uniquecount);


//sum of unique values
// let count=0;
// for (let val of newarr){
//     count+=val;
// }
// console.log(count);//15

//using reduce
// let count=newarr.reduce((count,val)=>{
//     return count+=val;
// })
// console.log(count);//15
```

```
//Check Duplicates
// function containsDuplicate(arr){
//     let uniqueValues=new Set(arr);
//     return uniqueValues.size!==arr.length;
// }
// console.log(containsDuplicate([1,2,3,4,5]));//false
// console.log(containsDuplicate([1,2,2,4,1]));//true

//Common Elements (intersection)
// function intersection(arr1,arr2){
//     let set1=new Set(arr1);
//     return arr2.filter(value=>set1.has(value));
// }
// let A=[1,2,3,4];
// let B=[3,4,5,6];
// console.log(intersection(A,B));//[ 3, 4 ]

//Union of Two Arrays
// function union(arr1,arr2){
//     return [...new Set([...arr1,...arr2])];
// }
// console.log(union([1,2,3],[3,4,5]));//[ 1, 2, 3, 4, 5 ]

//Difference Between Arrays
// function difference(arr1,arr2){
//     let set2=new Set(arr2);
//     return arr1.filter((value)=>!set2.has(value))
// }
// console.log(difference([1,2,3,4],[3,4]));//[ 1, 2 ]

//First Repeating Element
// function firstRepeating(arr){
//     let seen=new Set();
//    for(let value of arr){
//     if(seen.has(value)){
//         return value;
//     }
//     seen.add(value);
//    }
//    return null;
// }
// console.log(firstRepeating([4,5,1,2,5,1]));//5

//Unique Words in Sentence or remove duplicate words
// function unique(sentence){
//     let words=sentence.split(" ");
//     let uniqueWords=[...new Set(words)];
//     return uniqueWords.join(" ");
// }
// let text="this is is a test test";
// console.log(unique(text));//this is a test

//Longest Substring Without Repeating Characters
//Longest Consecutive Sequence
```

```
//Check If All Elements Are Unique
// function uniqueElement(arr){
//     let arr2=new Set(arr);
//     return (arr2.size===arr.length);
//     }
// console.log(uniqueElement([1,2,3,4]));//true
// console.log(uniqueElement([1,2,2,4]));//false

//Find Missing Numbers in Range
// function missingNum(arr,n){
//     let arr2=new Set(arr);
//     let missing=[];
//     for(let i=1;i<=n;i++){
//       if(!arr2.has(i)){
//          missing.push(i);
//       }
//     }
//     return missing;
// }
// console.log(missingNum([1,2,4,6],6));//[ 3, 5 ]

//Find Symmetric Difference
// function symmetricDiff(arr1,arr2){
//     let set1=new Set(arr1);
//     let set2=new Set(arr2);
//     let result=[];
//     for(let val of set1){
//         if(!set2.has(val)){
//             result.push(val);
//         }
//     }
//     for(let val of set2){
//         if(!set1.has(val)){
//             result.push(val);
//         }
//     }
//     return result;
// }
// let A=[1,2,3];
// let B=[3,4,5];
// console.log(symmetricDiff(A,B));//[ 1, 2, 4, 5 ]

                                //MAP**********//
//Create and Print Map
// const details=new Map([
//     ["name","Bhakti"],
//     ["age",22],
//     ["city","Pune"]
// ]);
// console.log(details);//Map(3) { 'name' => 'Bhakti', 'age' => 22, 'city' => 'Pune' }

//using set and get
// const details=new Map();
```

```
// details.set("name","Bhakti");
// details.set("age",22);
// details.set("city","Pune");
// console.log(details);
// console.log(details.get("name"));//Bhakti
// for(let [key,value] of details){
//     console.log(key,value);
// }  //name Bhakti
     // age 22
     // city Pune

// details.forEach((value,key)=>{
//     console.log(key,value);
// })   //name Bhakti
       // age 22
       // city Pune

//Check Key Exists
// let fruits = new Map([
//   ["apple", 10],
//   ["banana", 20]
// ]);
// console.log(fruits.has("apple"));//true
// if(fruits.has("apple")) {
//     console.log(fruits.get("apple"))//10
// }
// console.log(fruits.has("mango"));//false
// console.log(typeof fruits);//object

//Count Frequency of Numbers
// const arr = [1,2,2,3,3,3,4];
// let freq=new Map();
// for(let i=0;i<arr.length;i++){
//     let num=arr[i];
//     if(freq.has(num)){
//         freq.set(num,freq.get(num)+1);
//     }else{
//         freq.set(num,1);
//     }
// }
// console.log(freq);//Map(4) { 1 => 1, 2 => 2, 3 => 3, 4 => 1 }

//First Non-Repeating Character
// function firstNonRepeating(str){
//     let freq=new Map();
//     for(let ch of str){
//         freq.set(ch,(freq.get(ch)||0)+1);
//     }
//     for(let ch of str){
//         if(freq.get(ch)===1){
//             return ch;
//         }
//     }
//     return null;
```

```
// }
// console.log(firstNonRepeating("programming"));//p

//Find Duplicates in Array or Find All Elements That Appear Exactly Twice
// const arr=[1,2,3,4,2,3,5];
// let freq=new Map();
// let duplicates=[];
// for(let num of arr){
//     freq.set(num,(freq.get(num) || 0)+1);
//     if(freq.get(num)===2){
//         duplicates.push(num);
//     }
// }
// console.log(duplicates);//[ 2, 3 ]

//Find Most Frequent Element
// function mostFreq(arr){
//      let freq=new Map();
//      for (let num of arr){
//           freq.set(num,(freq.get(num)||0)+1);
//      }
//      let max=0;
//      let maxNum;
//      for(let [num,count] of freq){
//           if(count>max){
//                max=count;
//                maxNum=num;
//           }
//      }
//      return maxNum;
// }
// console.log(mostFreq([1,1,2,3,3,3,4]));//3

//Group Words by Length
// function groupByLength(words){
//      let map=new Map();
//      for(let word of words){
//           let len=word.length;
//           if(!map.has(len)){
//                map.set(len,[]);
//           }
//           map.get(len).push(word);
//      }
//      return map;
// }
// console.log(groupByLength(["hi", "hello", "cat", "dog", "javascript"]
//  ));   //      Map(4) {
          //      2 => [ 'hi' ],
          //      5 => [ 'hello' ],
          //      3 => [ 'cat', 'dog' ],
          //      10 => [ 'javascript' ]
          //      }

//Two Sum (Using Map)
```

```
// function twoSum(num,target){
//      let map=new Map();
//      for(let i=0;i<num.length;i++){
//       let complement=target-num[i];
//       if(map.has(complement)){
//           return [map.get(complement),i];
//       }
//       map.set(num[i],i);
//      }
//      return [];
// }
// console.log(twoSum([2,7,11,15],9));//[ 0, 1 ]

//Count Characters in Each Word
// function countChar(words){
//      let map = new Map();
//      for(let word of words){
//           let count=word.length;
//           map.set(word,count);
//      }
//      return map;
// }
// console.log(countChar(["apple", "banana"]));//Map(2) { 'apple' => 5, 'banana' => 6 }

//Are Two Strings Anagrams?
// function areAnagram(str1,str2){
// if(str1.length!==str2.length){
// return false;
// }
// let map=new Map();
// for(let char of str1){
//      map.set(char,(map.get(char)||0)+1);
// }
// for(let char of str2){
// if(!map.has(char)){
//      return false;
// }
//      map.set(char,(map.get(char))-1);
// }
// for(let value of map.values()){
//      if(value!==0){
//           return false;
//      }
// }
// return true;
// }
// console.log(areAnagram("listen","silent"));//true

//Group Anagrams
// function groupAnagrams(arr){
//      let map=new Map();
//       for(let word of arr){
//           let key=word.split("").sort().join("");
//           if(map.has(key)){
```

```
//                map.get(key).push(word);
//            }else{
//                map.set(key,[word]);
//            }
//        }
//        return Array.from(map.values());
// }
// console.log(groupAnagrams(["eat","tea","tan","ate","nat","bat"]));
//[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

//Intersection of Two Arrays
// function intersectionArray(arr1,arr2){
//      let map=new Map();
//      let result=[];
//      for(let num of arr1){
//       map.set(num,(map.get(num)||0)+1);
//      }
//      for(let num of arr2){
//           if(map.has(num)&&map.get(num)>0){
//           result.push(num);
//           map.set(num,(map.get(num)||0)-1);
//       }
//      }
//      return result;
// }
// console.log(intersectionArray([1,2,2,1], [2,2]));//[ 2, 2 ]

//Count Elements Greater Than K
// function greaterElement(arr,k){
//      let map=new Map();
//      for(let num of arr){
//           if(num>k){
//             map.set(num,(map.get(num)||0)+1);
//           }
//      }
//      return map;
// }
// console.log(greaterElement([3,7,1,9,3,7,5],5));//Map(2) { 7 => 2, 9 => 1 }

//Find First Unique Number
// function firstUniqueNum(arr){
//      let map=new Map();
//      for(let num of arr){
//           map.set(num,(map.get(num)||0)+1);
//      }
//      for(let num of arr){
//           if(map.get(num)===1){
//                return num;
//           }
//      }
//      return null;
// }
// console.log(firstUniqueNum([4,5,1,2,0,4]));//5
```

```
//find all unique numbers
// function allUniqueNum(arr){
//      let map=new Map();
//      let result=[];
//      for(let num of arr){
//           map.set(num,(map.get(num)||0)+1);
//      }
//      for(let num of arr){
//           if(map.get(num)===1){
//                result.push(num);
//           }
//      }
//      return result;
// }
// console.log(allUniqueNum([4,5,1,2,0,4]));//[ 5, 1, 2, 0 ]

//Find the Element That Appears More Than n/3 Times
// function elementAppear(arr){
//      let map=new Map();
//      let result=[];
//      for(let num of arr){
//           map.set(num,(map.get(num)||0)+1);
//      }
//      let limit=Math.floor(arr.length/3);
//      for(let [num,count] of map.entries()){
//           if(count>limit){
//                result.push(num);
//           }
//      }
//      return result;
// }
// console.log(elementAppear([1,1,1,3,3,2,2,2]));//[ 1, 2 ]

               //********* DESTRUCTURING
     //Array Destructuring Problems
//Swap Two Variables
// let a=5,b=10;
// [a,b]=[b,a];
// console.log(a,b);//10 5

//Skip Elements
// let arr = [1, 2, 3, 4, 5];// get first and third
// let [a,,d]=arr;
// console.log(a,d);//1 3

//Rest Operator with Destructuring
// let numbers = [10, 20, 30, 40, 50];
// let [a,b,...c]=numbers;
// console.log([a,b]);//[ 10, 20 ]
// console.log(c);//[ 30, 40, 50 ]

     //Object Destructuring Problems
//Simple Object Destructuring
// let person = { name: "Alice", age: 25, city: "Paris" };
```

```
// let{name,age}=person;
// console.log(name,age);//Alice 25

//Rename Variables
// let person = { name: "Bob", age: 30, city: "London" };
// let {name:userName,age:userAge}=person;
// console.log(userName,userAge);//Bob 30

//Default Values
// let person = { name: "Charlie", age: 22};
//  let {name,age,country="India"}=person;
//  console.log(country);//India

// let person = { name: "Charlie", age: 22,country:"USA"};
//  let {name,age,country="India"}=person;
//  console.log(country);//USA

// let person2 = { name: "Charlie", age: 22, country: "USA" };
// let { name: n, age: a, country: c = "India" } = person2;
// console.log(c); // USA

     // Nested Destructuring Problems
 //Object inside Object
//  let person = {
//   name: "Dave",
//   address: {
//     city: "New York",
//     zip: 10001
//   }
// };
// let {name,address:{city}}=person;
// console.log(name,city);//Dave New York

//Array inside Object
// let person = {
//   name: "Eve",
//   skills: ["JavaScript", "React", "Node"]
// };
// let {skills:[a,b]}=person;
// console.log(a,b);//JavaScript React

//Nested Arrays
// let matrix = [[1,2,3],[4,5,6],[7,8,9]];
// let [[a,,],[b,,],[c,,]]=matrix;
// console.log(a,b,c);//1 4 7

// Mixed Destructuring
//Extract:name from object first skill from skills array country with default "USA"
// let person = {
//   name: "Frank",
//   skills: ["HTML", "CSS", "JS"]
// };
// let {name,skills:[a],country="USA"}=person;
// console.log(name,a,country);//Frank HTML USA
```

```
            //******** SPREAD OPERATOR(...):-used to expand an iterable (like an array or
object) into individual elements.
            // *******REST OPERATOR(...):-used to collect multiple elements or properties
into a single variable.
  // Array Spread & Rest Problems
// let arr1 = [1, 2];
// let arr2 = [3, 4, 5];
// let result=[...arr1,...arr2];
// console.log(result);//[ 1, 2, 3, 4, 5 ]

//Copy an Array
// let arr=[10,20,30];
// let copyarr=[...arr,40];
// console.log(arr);//[ 10, 20, 30 ]
// console.log(copyarr);//[ 10, 20, 30, 40 ]

//Skip & Gather
// let numbers = [5,10,15,20,25];
// let[a,b,...c]=numbers;
// console.log(a,b);//5 10
// console.log(c);//[ 15, 20, 25 ]

//Max Using Spread
// let nums = [12, 5, 20, 8];
// let max=Math.max(...nums);
// console.log(max);//20

   //Object Spread & Rest Problems
//Merge Two Objects
// let obj1 = {a:1, b:2};
// let obj2 = {b:3, c:4};
// let mergeobj={...obj1,...obj2};
// console.log(mergeobj);//{ a: 1, b: 3, c: 4 }
//the last value of b is selected so b value is overriden

//Copy Object
// let user = {name:"Alice", age:25};
// let copyUser={...user};
// console.log(user);
// console.log(copyUser);

//Extract & Gather
// let person = {name:"Bob", age:30, city:"London"};
// let {name,...b}=person;
// console.log(name);//Bob
// console.log(b);//{ age: 30, city: 'London' }

   // Function Parameters with Rest
//Sum Any Number of Arguments
// function sum(...nums){
//      let result=0;
//      for(let num of nums){
//           result+=num;
```

```
//      }
//      return result;
// }
// console.log(sum(...[1,2,3,4]));//10
// console.log(sum(...[5,10]));//15

// Nested Object Spread
// let obj1 = {a:1, b:{x:10,y:20}};
// let obj2 = {b:{y:50,z:30}, c:3};
// let mergeObj={...obj1,...obj2,b:{...obj1.b,...obj2.b}};
// console.log(mergeObj);//{ a: 1, b: { x: 10, y: 50, z: 30 }, c: 3 }

//Multiply first & Return Rest
// function process(first,...rest){
//      let multiply=first*2;
//      return [multiply,rest];
// }
// console.log(process(2,3,4,5));//[ 4, [ 3, 4, 5 ] ]

//Combine Rest & Destructuring
// let skills = ["JS","React","Node"];
// let[a,...b]=skills;
// console.log(a);//JS
// console.log(b);//[ 'React', 'Node' ]

//Merge Arrays & Remove Duplicates
// let arr1 = [1,2,3];
// let arr2 = [2,3,4];
// let merge=[...arr1,...arr2];
// let set=new Set(merge);
// console.log(set);//Set(4) { 1, 2, 3, 4 }
// let result=[...set];
// console.log(result);//[ 1, 2, 3, 4 ]


         //*********FUNCTION BORROWING
  //call()
// let obj1 = {
//   name: "Aman",
//   greet: function(city) {
//     console.log("Hello " + this.name + " from " + city);
//   }
// };

// let obj2 = {
//   name: "Riya"
// };

// // Use call() to make obj2 use greet()
// obj1.greet.call(obj2,"Bidar");//Hello Riya from Bidar

//Two Parameters
// let user1 = {
//   name: "Karthik"
```

```
// };

// function introduce(age, profession) {
//   console.log(this.name + " is " + age + " years old and works as " + profession);
// }

// let user2 = {
//   name: "Priya"
// };
// introduce.call(user2,21,"engineer");//Priya is 21 years old and works as engineer
// introduce.call(user1,21,"engineer");//Karthik is 21 years old and works as engineer

//Borrow Method Between Objects
// let student1 = {
//   name: "Nikhil",
//   details: function(marks) {
//     console.log(this.name + " scored " + marks);
//   }
// };

// let student2 = {
//   name: "Ananya"
// };
// // Borrow details using call()
// student1.details.call(student2,90);//Ananya scored 90

//Changing Context
// function showCountry(country) {
//   console.log(this.name + " is from " + country);
// }
// let personA = { name: "Rohit" };
// let personB = { name: "Sneha" };
// // Call showCountry for both objects using call()
// showCountry.call(personA,"INDIA");//Rohit is from INDIA
// showCountry.call(personB,"USA");//Sneha is from USA

// Multiple Calls
// let company1 = { name: "Infosys" };
// let company2 = { name: "Wipro" };

// function companyInfo(location, employees) {
//   console.log(this.name + " is located in " + location +
//               " and has " + employees + " employees");
// }
// // Use call() for both company1 and company2
// companyInfo.call(company1,"banglore",2000);//Infosys is located in banglore and has 2000
employees
// companyInfo.call(company2,"hyderabad",1000);//Wipro is located in hyderabad and has 1000
employees

  //apply()
// let person1 = {
//   name: "Aman"
// };
```

```
// function greet(city) {
//   console.log("Hello " + this.name + " from " + city);
// }

// let person2 = {
//   name: "Riya"
// };
// // Use apply() so person2 can use greet
// greet.apply(person1,["Bidar"]);//Hello Aman from Bidar
// greet.apply(person2,["banglore"]);//Hello Riya from banglore

//Multiple Parameters
// let user1 = {
//   name: "Karthik"
// };

// function introduce(age, profession) {
//   console.log(this.name + " is " + age + " years old and works as " + profession);
// }
// let user2 = {
//   name: "Priya"
// };
// introduce.apply(user1,[21,"engineer"]);//Karthik is 21 years old and works as engineer
// introduce.apply(user2,[23,"Doctor"]);//Priya is 23 years old and works as Doctor

//Method Borrowing Between Objects
// let student1 = {
//   name: "Nikhil",
//   details: function(marks, grade) {
//     console.log(this.name + " scored " + marks + " and got grade " + grade);
//   }
// };
// let student2 = {
//   name: "Ananya"
// };
// student1.details.apply(student2,[90,'A']);//Ananya scored 90 and got grade A

//Math Borrowing
// let numbers = [10, 20, 5, 40];
// let max=Math.max.apply(null,numbers);
// console.log(max);//40

/*call(), apply(), and bind() do NOT work with arrow functions because arrow functions don’t
have their own this.
   arrow functions capture this from their lexical (surrounding) scope.*/

   //bind()-returns a new function,does not execute immediately
// let person1 = {
//   name: "Aman",
//   greet: function(city) {
//     console.log(this.name + " says hello from " + city);
//   }
// };
```

```
// let person2 = {
//   name: "Riya"
// };
// let greetPerson2=person1.greet.bind(person2,"Bidar");
// greetPerson2();//Riya says hello from Bidar

//Bind with Multiple Arguments
// let user = {
//   name: "Karthik"
// };
// function introduce(age, profession) {
//   console.log(this.name + " is " + age + " years old and works as " + profession);
// }
// let newUser = { name: "Priya" };
// let introduceUser=introduce.bind(newUser,23,"Doctor");
// introduceUser();//Priya is 23 years old and works as Doctor

//Bind for Method Borrowing
// let student1 = {
//   name: "Nikhil",
//   details: function(marks, grade) {
//     console.log(this.name + " scored " + marks + " and got grade " + grade);
//   }
// };
// let student2 = { name: "Ananya" };
// let studentDetails=student1.details.bind(student2,90,'A');
// studentDetails();//Ananya scored 90 and got grade A

//Bind + setTimeout
// let obj = {
//   name: "Rahul",
//   greet: function() {
//     console.log("Hello " + this.name);
//   }
// };
// setTimeout(()=>{
//    let greetUser=obj.greet.bind(obj)
//    greetUser();
// },1000);//Hello Rahul

//OR
//setTimeout(obj.greet.bind(obj), 1000);

//Bind Partial
// function multiply(a, b) {
//   console.log(a * b);
// }
// let multiplyBy2 = multiply.bind(null, 2);
// multiplyBy2(5);//10
// multiplyBy2(10);//20

    //********Template Literals
/*Use backticks ` for template literals.
```

```
${expression} allows dynamic interpolation.
Supports multi-line strings without \n.
Can embed variables, math, functions, objects, arrays inside ${}.*/

//Basic Interpolation
// let firstName = "Riya";
// let lastName = "Sharma";
// console.log(`Hello, ${firstName} ${lastName}! Welcome to Javascript`);
// //Hello, Riya Sharma! Welcome to Javascript

//Expression Inside Template Literals
// let a = 7;
// let b = 3;
// let result=`The product of ${a} and ${b} is ${a*b}`;
// console.log(result);//The product of 7 and 3 is 21

//Object Property Inside Template Literal
// let user = { name: "Aman", age: 21 };
// console.log(`${user.name} is ${user.age} years old`);
//Aman is 21 years old

//Array Element Inside Template Literal
// let fruits = ["Apple", "Banana", "Mango"];
// console.log(`My favourite fruits are ${fruits[0]}, ${fruits[1]}, and ${fruits[2]}`);
//My favourite fruits are Apple, Banana, and Mango

//Function Call Inside Template Literal
// function greet(name) {
//   return `Welcome ${name}`;
// }
// console.log(`${greet("Riya")} to the Javascript course!`);
//Welcome Riya to the Javascript course!

//Multi-line String Using Template Literal
// console.log(`Hello, World!
// This is a multi-line string.
// Enjoy coding!`);

     //***********Optional Chaining (?.) ,  Nullish Coalescing (??)

/*Use ?.() for safe function calls
Use ?. for safe property access
Use ?? to provide default values*/

//Access nested object property safely
// let user = {
//   name: "Riya",
//   address: { city: "Bidar" }
// };
// Print the city if it exists, otherwise print "City not available"
//console.log(user.address?.city??"City not available");

//Function call safely
// let user2 = {
```

```
//   name: "Aman",
//   greet: () => "Hello!"
// };
// // Call greet function safely and print it. If greet doesn't exist, print "No greeting"
// console.log(user2.greet?.()??"No greeting");

//Optional array element
// let fruits = ["Apple", "Banana"];
// // Print the 3rd fruit if it exists, otherwise print "Fruit not found"
// console.log(fruits?.[2]??"fruit not found");

//Nested object with default value
// let student = {
//   name: "Nikhil",
//   scores: null
// };
// // Print the math score if it exists, otherwise print 0
// console.log(student.scores?.math??0);

//Combine optional chaining and nullish coalescing
// let company = { name: "Infosys", location: { city: null } };
// // Print the city if exists, otherwise print "Unknown City"
// console.log(company.location?.city??"Unknown City");

      //CLOSURE********
/*A function that remembers variables from its outer scope even after the outer
function has finished executing.*/

// function counter() {
//   let count = 0;

//   return function() {
//     count++;
//     return count;
//   };
// }

// let c1 = counter();
// let c2 = c1;
// console.log(c1());//1
// console.log(c2());//2


// let a = counter();
// let b = counter();
// console.log(a());//1
// console.log(a());//2
// console.log(b());//1
// console.log(a());//3

// function createName(){
//      let name="Bhakti";
//      return function(){
//           return name;
```

```
//      }
// }
// let fn =createName();
// console.log(fn());//Bhakti

// function createCounter(){
//      let count=0;
//      return function(){
//           count++;
//           return count;
//      }
// }
// let fn1=createCounter();
// let fn2=createCounter();
// console.log(fn1());//1
// console.log(fn1());//2
// console.log(fn2());//1
// console.log(fn1());//3
// console.log(fn2());//2
// let fn3=fn2;
// console.log(fn3());//3

// function createCounter(){
//      let count=0;
//      const obj={
//           increment(){
//              count++;
//              return count;
//           },
//           decrement(){
//                count--;
//                return count;
//           },
//           reset(){
//                count=0;
//                return count;
//           }
//      }
//      return obj;
// }
// let counter=createCounter();
// console.log(counter.increment());//1
// console.log(counter.increment());//2
// console.log(counter.decrement());//1
// console.log(counter.reset());//0
// console.log(counter.increment());//1

// function once(fn) {
//    let called = false;

//    return function() {
//        if (!called) {
//             called=true;
//            return fn()
```

```
//         }
//     }
// }
// let greet = once(() => console.log("Hello"));

// greet(); // Hello
// greet(); // nothing
// greet(); // nothing

// function createBankAccount(initialBalance){
//      let total=initialBalance;
//      let obj={
//           deposit(amount){
//                total+=amount;
//                return total;
//           },
//           withdraw(amount){
//                if(amount > total){
//                return "Insufficient balance";
//           }
//                total -= amount;
//                return total;
//           },
//           getBalance(){
//                return total;
//           }
//      }
//      return obj;
// }
// let account=createBankAccount(1000);
// account.deposit(500);
// account.withdraw(200);
// console.log(account.getBalance());//1300

// function multiplyBy(x){
//      return function(y){
//           return x*y;
//      }
// }
// let double=multiplyBy(2);
// let triple=multiplyBy(3);
// console.log(double(5));//10
// console.log(triple(5));//15

// function createLogger(){
//      let arr=[];
//      let obj={
//           log(message){
//                arr.push(message);
//           },
//           getLogs(){
//                return [...arr];
//           }
//      }
```

```
//      return obj;
// }
// let logger=createLogger();
// logger.log("Hello");
// logger.log("World");
// console.log(logger.getLogs());//[ 'Hello', 'World' ]

/*Countdown (n → 1): (n - i) * 1000
Countup (1 → n): (i - 1) * 1000*/

// function countdownLogger(n){
//      return {
//           start(delay=0){
//         for(let i=n;i>0;i--){
//           setTimeout(function(){
//             console.log(i);
//           },delay+(n-i)*1000);
//         }
//         setTimeout(function(){
//           console.log("Done!");
//         },delay+n*1000);
//      }
// }
// }
// let counter1 = countdownLogger(3);
// let counter2 = countdownLogger(5);
// counter1.start();
// counter2.start(4000);

// function advancedCounter(n) {
//     let current = n;
//     let timerId = null;
//     let isPaused = false;

//     function runCountdown() {
//         if (current > 0 && !isPaused) {
//             console.log(current);
//             current--;
//             timerId = setTimeout(runCountdown, 1000);
//         } else if (current === 0) {
//             console.log("Done!");
//         }
//     }

//     return {
//         start() {
//             if (timerId === null) { // prevent multiple starts
//                 isPaused = false;
//                 runCountdown();
//             }
//         },
//         pause() {
//             if (timerId !== null) {
//                 clearTimeout(timerId);
```

```
//                 isPaused = true;
//             }
//         },
//         resume() {
//             if (isPaused && current > 0) {
//                 isPaused = false;
//                 runCountdown();
//             }
//         },
//         reset() {
//             clearTimeout(timerId);
//             current = n;
//             timerId = null;
//             isPaused = false;
//         }
//     };
// }
// let counter = advancedCounter(5);
// counter.start();   // starts 5 → 4 → 3 → 2 → 1 → Done!
// setTimeout(() => counter.pause(), 2500);   // pauses after 2.5 sec
//setTimeout(() => counter.resume(), 4000);  // resumes after 4 sec
//setTimeout(() => counter.reset(), 8000);   // resets after 8 sec

                    //*******CALLBACKS
//Write a function calculate(a, b, operation)
// function calculate(a,b,operation){
//      return operation(a,b);
// }
// function add(x,y){
//      return x+y;
// }
// function subtract(x,y){
//      return x-y;
// }
// console.log(calculate(10,8,add));
// console.log(calculate(10,8,subtract));

// function greetUser(name,callback){
//      callback(name);
// }
// function greet(name){
//      console.log("Hi " +name);
// }
// greetUser("Bhakti",greet);

// //Use setTimeout to print:
// console.log("Loading...");
// setTimeout(function(){
//      console.log("Done!")
// },3000);

//forEach
// function myForEach(arr,callback){
//      for (let i = 0; i < arr.length; i++) {
```

```
//         callback(arr[i], i, arr);
//     }
// }
// myForEach([1,2,3,4,5],function(num){
//         console.log(num);
//      });

// function fetchData(callback){
// setTimeout(function(){
//      const details={id:1,name:"Product"};
//      callback(details);
// },2000);
// }
// fetchData(function(data){
//      console.log(data);
// })

// let arr=[1,2,3,4];
// arr.forEach(function(num){
//      console.log(num*2);
// })

//OR

// function myMap(arr, callback) {
//     let result = [];
//     for (let i = 0; i < arr.length; i++) {
//         result.push(callback(arr[i], i, arr));
//     }
//     return result;
// }
// let output = myMap([1,2,3], function(num) {
//     return num*2;
// });
// console.log(output);

// function doTask(taskName,callback){
//      console.log("Starting" + taskName);
//      setTimeout(function(){
//           console.log("Finished" + taskName);
//           if(callback){
//                callback();
//           }
//      },2000);
// }
// doTask("Task 1",function(){
//      doTask("Task 2", function(){
//           doTask("Task 3", function(){
//                console.log("All tasks completed");
//           });
//      });
// });

// function login(username,password,callback){
```

```
//     setTimeout(function(message){
//      if(username==="admin" && password==="1234"){
//           callback("Login Success");
//      }else{
//           callback("Login Failed");
//      }
//     },2000)
// }
// login("admin","1234",function(message){
//    console.log(message);
// });

/*function getUser(userId, callback) {
    setTimeout(() => {
        if (userId === 1) {
            callback(null, { id: 1, name: "Bhakti" });
        } else {
            callback("User Not Found", null);
        }
    }, 1000);
}

function getOrders(user, callback) {
    setTimeout(() => {
        callback(null, ["Shoes", "Laptop"]);
    }, 1000);
}

function checkStock(orders, callback) {
    setTimeout(() => {
        let inStock = true; // change to false to test
        if (inStock) {
            callback(null, "Stock Available");
        } else {
            callback("Out of Stock", null);
        }
    }, 1000);
}

function makePayment(orders, callback) {
    setTimeout(() => {
        callback(null, "Payment Successful");
    }, 1000);
}

getUser(1,function(error,user){
     if(error){
          console.log(error);
          return;
     }
     console.log(user);
     getOrders(user,function(error,order){
          if(error){
          console.log(error);
```

```
          return;
     }
     console.log(order);
     checkStock(order,function(error,stock){
          if(error){
          console.log(error);
          return;
     }
     console.log(stock);
     makePayment(order,function(error,result){
          if(error){
          console.log(error);
          return;
     }
     console.log(result);
     });
     });
     });
});*/

            //*******PROMISES
/*let myPromise = new Promise(function(resolve, reject){
    let success = true;
    if(success){
        resolve("Task Completed");
    } else {
        reject("Task Failed");
    }
});

myPromise
   .then(function(result){
       console.log(result);
   })
   .catch(function(error){
       console.log(error);
   });*/

/*let myPromise=new Promise(function(resolve,reject){
    let isError=true;
    setTimeout(()=>{
    if(isError){
        reject("Loading Failed");
    }else{
        resolve("Data Loaded");
    }
    },2000)
})
myPromise.then(function(result){
    console.log(result);
})
.catch(function(result){
    console.log(result);
})*/
```

```
/*function getUser(){
    return new Promise(function(resolve,reject){
        setTimeout(()=>{
            let userId=1;
            if(userId===1){
                resolve({id:1,name:"Bhakti"});
            }else{
                reject("User Not Found");
            }
        },1000)
    })
}
function getOrders(user){
    return new Promise(function(resolve,reject){
        setTimeout(()=>{
         if(user && user.id){
            resolve(["Shoes","Laptop"]);
         }else{
            reject("Invalid User");
         }
        },1000)
    })
}
function makePayment(orders){
    return new Promise(function(resolve,reject){
        setTimeout(()=>{
         if(orders.length>0){
            resolve("Payment Successful");
         }else{
            reject("No Orders Found");
         }
        },1000)
    })
}
getUser().then(function(user){
console.log(user);
return getOrders(user);
})
.then(function(orders){
    console.log(orders);
    return makePayment(orders);
})
.then(function(payment){
    console.log(payment);
})
.catch(function(error){
    console.log(error);
})*/

/*Inside .catch():
return value → becomes resolved → next .then() runs
throw error → becomes rejected → next .then() skipped
Error → catch → if no throw → chain continues
```

```
Error → catch → if throw again → chain stops until next catch*/

/*function loginUser(username,password){
    return new Promise(function(resolve,reject){
        setTimeout(()=>{
            if(username==="Bhakti" && password==="1234"){
                resolve("Login Successful");
            }else{
                reject("Invalid Credentials");
            }
        },1000)
    })
}
function fetchProfile(){
    return new Promise(function(resolve,reject){
        setTimeout(()=>{
      let randomNum=Math.random();
      if(randomNum<0.7){
        resolve({name:"Bhakti",role:"Developer"});
      }else{
        reject("Profile Fetch Failed");
      }
        },1000)
    })
}
function logoutUser(){
    return new Promise(function(resolve,reject){
        setTimeout(()=>{
           resolve("Logged Out");
        },1000)
    })
}
loginUser("Bhakti","1234")
.then(function(login){
    console.log(login);
    return fetchProfile();
})
.then(function(profile){
    console.log(profile);
})
.catch(function(error){
    if(error === "Profile Fetch Failed"){
        console.log(error);
        return;
    } else {
        throw error;
    }
})
.then(function(){
    return logoutUser();
})
.then(function(logout){
    console.log(logout);
})
```

```
.catch(function(error){
    console.log(error);
});*/

/*If a promise resolves → next .then() runs.
If a promise rejects → skip all .then() → go to nearest .catch().
after .catch()?
Two possibilities:
 If catch RETURNS something:
Chain becomes resolved again
Next .then() WILL run
 If catch THROWS error:
Chain stays rejected
Next .then() will NOT run*/

/*| Situation           | What Happens     |
| ------------------- | ---------------- |
| throw inside then   | becomes rejected |
| return inside then  | becomes resolved |
| throw inside catch  | rejected again   |
| return inside catch | resolved again   |*/

                //**********Async/Await
/*async makes a function return a promise automatically
Even if you return a normal value.
await pauses execution until promise resolves
But it only works inside an async function.
Error handling uses try...catch
Instead of .then().catch()*/

/*| What you write | What JS converts it to   |
| -------------- | ------------------------ |
| `return value` | `Promise.resolve(value)` |
| `throw error`  | `Promise.reject(error)`  |*/

/*function loginUser(username, password) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (username === "Bhakti" && password === "1234") {
                resolve("Login Successful");
            } else {
                reject("Invalid Credentials");
            }
        }, 1000);
    });
}
function fetchProfile() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("Profile Data Loaded");
        }, 1000);
    });
}
function logoutUser() {
```

```
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("User Logged Out");
        }, 1000);
    });
}
async function run() {
    try {
        const login = await loginUser("Bhakti", "1234");
        console.log(login);

        const profile = await fetchProfile();
        console.log(profile);

        const logout = await logoutUser();
        console.log(logout);

    } catch (error) {
        console.log("Error:", error);
    }
}
run();*/

/*function registerUser(name){
    return new Promise((resolve,reject)=>{
        setTimeout(()=>{
           if(!name){
           reject("Name required")
           }else{
           resolve("User Registered")
           }
        },1000);
    })
}
function sendEmail(){
    return new Promise((resolve)=>{
        setTimeout(()=>{
          resolve("Email Sent");
        },1000)
    })
}
function logIn() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("Login Successful");
        }, 1000);
    });
}
async function run(){
    try{
        const register = await registerUser("Bhakti");
        console.log(register);

        const email=await sendEmail();
```

```
        console.log(email);

        const login=await logIn();
        console.log(login);

    }catch(error){
        console.log("Error:",error);
    }
}
run();*/

        //********Promise.all
/*It runs multiple promises in parallel and waits until all of them finish.
Promise.all([promise1, promise2, promise3])
It returns:Array of results (if ALL succeed) Rejects immediately (if ANY one fails)*/

/*async function run() {
    try {
        const results = await Promise.all([
            registerUser("Bhakti"),
            sendEmail(),
            logIn()
        ]);

        console.log(results);

    } catch (error) {
        console.log("Error:", error);
    }
}

run();*/

    //****Promise.allSettled
/*Waits for all promises to finish, regardless of success or failure
Returns an array of objects showing status and value/reason
used when You want all results, even if some fail
Promise.allSettled([p1, p2, p3])*/

    //****Promise.race
/*Runs multiple promises in parallel
Resolves/rejects as soon as the first promise finishes
Returns the value or reason of that first finished promise
Promise.race([p1, p2, p3])
used when First response matters*/

    //*****Promise.any
/*Runs multiple promises in parallel
Resolves with first fulfilled
Rejects only if all promises fail
Promise.any([p1, p2, p3])*/

/*function checkInventory(products) {
    return new Promise((resolve, reject) => {
```

```
        setTimeout(() => {
            const outOfStock = products.includes("Laptop") ? false : true;
            if (!outOfStock) {
                resolve(products);
            } else {
                reject("Some products are out of stock ");
            }
        }, 1000);
    });
}

function processPayment(amount) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const success = Math.random() < 0.7; // 70% chance
            if (success) {
                resolve("Payment Successful ");
            } else {
                reject("Payment Failed ");
            }
        }, 1000);
    });
}

function sendEmail(user) {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve(`Email sent to ${user} `);
        }, 1000);
    });
}

function deliveryOption(name, time) {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve(name);
        }, time);
    });
}

async function checkout() {
    try {
        const products = await checkInventory(["Shoes", "Laptop"]);
        console.log("Inventory OK:", products);

        const payment = await processPayment(500);
        console.log(payment);
        const fastestDelivery = deliveryOption("FedEx", 1500);
        const secondDelivery = deliveryOption("DHL", 1000);
        const thirdDelivery = deliveryOption("UPS", 2000);

        const [email, delivery] = await Promise.all([
            sendEmail("Bhakti"),
            Promise.race([fastestDelivery, secondDelivery, thirdDelivery])
```

```
        ]);

        console.log(email);
        console.log("Fastest delivery:", delivery);

        console.log("Checkout Complete ");

    } catch (error) {
        console.log("Error:", error);
    }
}
checkout();*/
```